

```
String code = "";
java.util.Random rd; int cd;
rd= new java.util.Random();
cd = rd.nextInt(100000);
code = String.valueOf(cd);
/*<To be continued...>*/
}
else
javaw.swing.JOptionPane.showMessageDialog(null,"Please enter
correct details");
}
```

The JavaMail API

You've now reached a point where you have to take a short detour. For your next step, you need something through which you could send an email to the Way2SMS ID using Java. Yes, the JavaMail API, the fourth and last ingredient. Using this API, you can write programs to send, receive and manage emails very efficiently in Java. The developers of this API have designed it so beautifully that application developers only have to write a few lines of code to get it running. It's absolutely free to download from <http://goo.gl/C38nv> and is under active development. Download version 1.4.5, which is the latest at the time of writing this article. Follow these simple steps to integrate it with your *Two Step Ver* project:

1. Extract the zip file to the project folder (TSV in this case).
2. Switch to the NetBeans window and right-click *Two Step Ver* in the *Projects* panel and choose the *Properties* option.
3. In the *Project Properties* window, select *Libraries* under *Categories* and click *Add JAR/Folder*
4. In the *Add JAR/Folder* window, navigate to the JavaMail API folder, select *mail.jar* and click *Open*.
5. Click *OK* in the *Project Properties* window, and that's it—you're done.

Returning to the code—you need to add code to send the randomly generated code to the registered cell phone number through SMS. First add these lines at the beginning, as shown below:

```
package tsv;

import javax.mail.*;
import javax.mail.internet.*;

Now, replace /*<To be continued...>*/ with the
following code:

/*Setting the mail server properties*/
java.util.Properties props = new java.util.
Properties();
props.put("mail.smtp.host", "smtp.gmail.com");
props.put("mail.smtp.socketFactory.port", "465");
props.put("mail.smtp.socketFactory.class", "javax.net.
ssl.SSLSocketFactory");
```

```
props.put("mail.smtp.auth", "true");
props.put("mail.sptn.port", "465");

/*Creating a session and sending the code*/
Session session = Session.getDefaultInstance(props,
new javax.mail.Authenticator(){
protected PasswordAuthentication
getPasswordAuthentication() {
return new PasswordAuthentication("<Gmail
Username>", "<Password>");});
try {
Message message = new MimeMessage(session);
message.setFrom(new InternetAddress("<Gmail
ID>"));
message.setRecipients(Message.RecipientType.
TO, InternetAddress.parse("<Way2SMS ID>"));
message.setSubject(String.valueOf(cd));
message.setText("User Logged in ");
Transport.send(message);
} catch(MessagingException e){
throw new RuntimeException(e);
}

/*Verifying the entered code*/
String rCode = JOptionPane.showInputDialog(null,"Please enter the
verification code sent to your registered number?");
if(rCode.equals(code))JOptionPane.showMessageDialog(null, "Second
step of user verification complete");
else
JOptionPane.showMessageDialog(null, "Wrong code
entered.");
```

The above code uses Gmail as the email-sending service, because it is the most commonly used. However, you can customise it as per your requirements; all you have to do is change the server properties in the above code. Please note, before compiling this code replace value tags (like *<Gmail Username>*) with actual values—else, of course, the code won't work.

Finally, to compile and run this code, right-click *AuthenticationForm.java* in the *Projects* panel, and select *Run File*, or click the *Play* icon, or press *Shift+F6*. I have intentionally not shown the output, just to whet your curiosity and get you to try this for yourself.

Readers who wish to use this functionality in their applications are free to use the code shown here. It might need some customising based on your requirements, but otherwise, it's ready to use. If you face any issues regarding this tutorial, contact me at the email ID below; in your mail, do include details of the errors you encountered. **END** 

By: Uday Mittal

The author is a Java Applications Developer and likes to experiment with new technologies. He provides SME and personalised solutions and can be reached at mittal.ud@gmail.com.